

Universidad de San Carlos de Guatemala

Facultad de Ingeniería
Escuela de Ciencias y Sistemas

LABORATORIO - IPC 2

Sección: P | Angel Miguel García Urizar

Segundo Semestre 2025

**Sistema de Biblioteca con POO en
Python
MANUAL TÉCNICO**

Presentado por:

Wilson Manuel Santos Ajcot

Carné: 2019-07179

santosajcot95@gmail.com

Fecha de entrega: 15/08/2025

Guatemala, Guatemala

Contenido

Índice

Introducción Técnica	2
Propósito y Audiencia	2
Arquitectura y Metodología	2
Objetivos	2
Objetivo General	2
Objetivos Específicos	2
Especificaciones Técnicas	3
Requisitos de Software y Librerías	3
Arquitectura del Software	4
Estructura del Proyecto	4
Descripción de Módulos y Clases	5
Módulo <code>main.py</code> : Punto de Entrada	5
Paquete <code>models</code> : Estructuras de Datos	5
Paquete <code>services</code> : Lógica de Negocio	7
Paquete <code>gui</code> : Interfaz de Usuario	7
Conclusiones	9
Conclusiones Técnicas	9
Apéndices	9
Código Fuente Completo	9

Introducción Técnica

Propósito y Audiencia

Este manual técnico detalla la arquitectura de software, las decisiones de diseño y la implementación del **Sistema de Turnos Médicos**. Está dirigido a desarrolladores, personal de mantenimiento y evaluadores académicos que necesiten comprender la estructura interna del código, las estructuras de datos utilizadas y la lógica de negocio para fines de mantenimiento, extensión o auditoría.

Arquitectura y Metodología

El sistema fue desarrollado en Python bajo una arquitectura en capas, separando las responsabilidades en: **Modelos** (estructuras de datos), **Servicios** (lógica de negocio), **GUI** (interfaz de usuario) y **Utilitarios** (constantes). Se aplicó el paradigma de Programación Orientada a Objetos (POO) para garantizar la modularidad, el encapsulamiento y la cohesión del código, facilitando así su escalabilidad y mantenimiento.

Objetivos

Objetivo General

Desarrollar una aplicación de escritorio funcional para la gestión de turnos médicos, aplicando estructuras de datos dinámicas no nativas (cola con nodos enlazados), una interfaz gráfica interactiva con Tkinter y capacidades de visualización de datos con Graphviz.

Objetivos Específicos

- ▶ **Implementar una Cola Dinámica:** Construir desde cero una estructura de datos tipo Cola, basada en nodos enlazados, para gestionar el flujo de pacientes bajo el principio FIFO, sin utilizar las listas o colecciones nativas de Python.
- ▶ **Desarrollar una GUI con Tkinter:** Crear una interfaz gráfica de usuario intuitiva que permita a los usuarios interactuar con el sistema para registrar, atender y visualizar turnos.
- ▶ **Integrar Graphviz:** Utilizar la librería Graphviz para generar representaciones visuales en tiempo real del estado de la cola de pacientes y de las estadísticas del sistema.
- ▶ **Aplicar Lógica de Negocio:** Simular el cálculo de tiempos de espera y atención, demostrando la capacidad del sistema para manejar datos asociados a cada turno.

Especificaciones Técnicas

Requisitos de Software y Librerías

A continuación, se listan las dependencias de software y las versiones requeridas.

- ▶ **Sistema Operativo:** Compatible con Windows, macOS y distribuciones de Linux que soporten Python y Graphviz.
- ▶ **Lenguaje de Programación:** Python 3.6 o superior.
- ▶ **Librerías de Python Requeridas:**
 - **tkinter:** Para la construcción de la interfaz gráfica. Generalmente incluido en la instalación estándar de Python.
 - **Pillow:** Utilizada para el manejo y visualización de imágenes en la GUI. Se instala con `pip install Pillow`.
 - **graphviz:** Librería de Python para interactuar con el software Graphviz y generar los diagramas. Se instala con `pip install graphviz`.
- ▶ **Software Externo:**
 - **Graphviz:** Es indispensable tener el software Graphviz instalado a nivel de sistema operativo y su directorio `bin` agregado al `PATH` del sistema.

Arquitectura del Software

Estructura del Proyecto

El proyecto está organizado en una estructura de directorios modular que separa claramente las responsabilidades, mejorando la legibilidad y el mantenimiento del código.

```
/
|-- src/
|   |-- main.py
|   |-- gui/
|   |   '-- interfaz.py
|   |-- models/
|   |   |-- nodo.py
|   |   |-- cola.py
|   |   |-- paciente.py
|   |   '-- reporte.py
|   |-- services/
|   |   |-- turno_service.py
|   |   '-- graphviz_service.py
|   '-- utils/
|-- constantes.py
|-- reportes/
    '(Archivos generados por Graphviz)
```

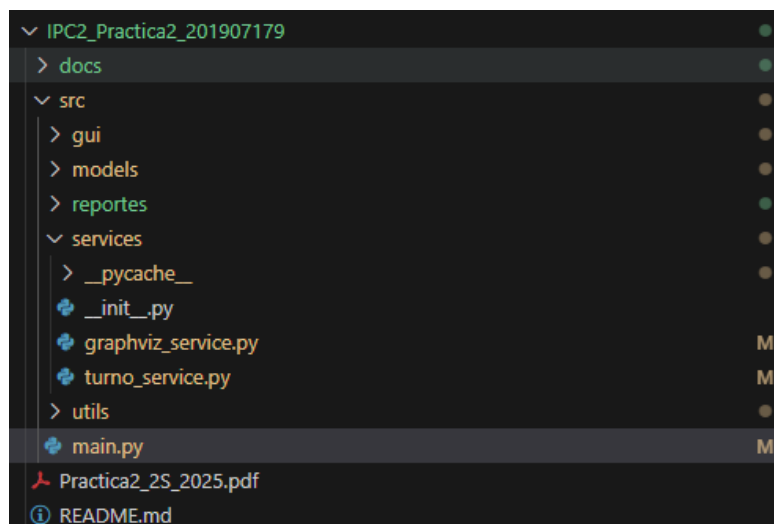


Figura 1: Estructura de directorios del proyecto.

Cada componente del sistema tiene un rol definido dentro de la arquitectura.

Módulo main.py: Punto de Entrada

Actúa como el lanzador de la aplicación. Sus responsabilidades se limitan a:

- Verificar las dependencias necesarias (`tkinter`, `Pillow`, `graphviz`).
- Mostrar información inicial del sistema en la consola.
- Instanciar y ejecutar la clase principal de la interfaz gráfica, `InterfazTurnos`.
- Gestionar errores fatales a un alto nivel.

[illegible]

Figura 2: Código del módulo main.py.

Paquete `models`: Estructuras de Datos

Este paquete es el núcleo del sistema, ya que contiene todas las estructuras de datos personalizadas.

- **nodo.py**: Define la clase `Nodo`, el bloque de construcción fundamental para las estructuras enlazadas. Contiene un dato y un puntero al siguiente nodo.
- **cola.py**: Implementa la clase `Cola`. Utiliza nodos para crear una estructura de datos FIFO dinámica. Contiene métodos como `encolar()`, `desencolar()` y la implementación del protocolo iterador (`__iter__` y `__next__`) para permitir recorridos sin usar listas nativas.
- **paciente.py**: Define la clase `Paciente`, que encapsula toda la información de un turno (nombre, edad, especialidad, hora de llegada).
- **reporte.py**: Contiene las clases `ReporteEstadisticas` y `NodoEstadistica`, diseñadas para transportar los datos de estadísticas desde la capa de servicio a la interfaz sin usar diccionarios.

```

# models/cola.py (USANDO COMENTARIOS)
"""
Módulo que implementa la estructura de datos Cola de forma dinámica.
No utiliza estructuras nativas de Python como list, deque, etc.
"""

from models.nodo import Nodo

class Cola:
    """
    Implementación de una cola dinámica usando nodos enlazados.
    Sigue el principio FIFO (First In, First Out).
    """

    def __init__(self):
        """Inicializa una cola vacía."""
        self.frente = None
        self.final = None
        self.tamaño = 0

    def esta_vacia(self):
        """Verifica si la cola está vacía."""
        return self.frente is None

    def encolar(self, dato):
        """Agrega un elemento al final de la cola."""
        nuevo_nodo = Nodo(dato)
        if self.esta_vacia():
            self.frente = nuevo_nodo
            self.final = nuevo_nodo
        else:
            self.final.siguiente = nuevo_nodo
            self.final = nuevo_nodo
        self.tamaño += 1

    def desencolar(self):
        """Remueve y retorna el elemento del frente de la cola."""
        if self.esta_vacia():
            raise IndexError("No se puede desencolar de una cola vacía")
        dato = self.frente.dato
        self.frente = self.frente.siguiente
        if self.frente is None:
            self.final = None
        self.tamaño -= 1
        return dato

    def ver_frente(self):
        """Retorna el elemento del frente sin removerlo."""
        if self.esta_vacia():
            raise IndexError("La cola está vacía")
        return self.frente.dato

    def obtener_tamano(self):
        """Retorna el número de elementos en la cola."""
        return self.tamaño

    def limpiar(self):
        """Purga completamente la cola."""
        self.frente = None
        self.final = None
        self.tamaño = 0

    def __str__(self):
        """Representación en cadena de la cola, CONSTRUIDA SIN USAR LISTAS."""
        if self.esta_vacia():
            return "Cola vacía"
        cadena_resultado = ""
        nodo_actual = self.frente
        while nodo_actual is not None:
            cadena_resultado += str(nodo_actual.dato)
            if nodo_actual.siguiente is not None:
                cadena_resultado += " -> "
            nodo_actual = nodo_actual.siguiente
        return cadena_resultado

    def __len__(self):
        """Retorna la longitud de la cola."""
        return self.tamaño

    def __iter__(self):
        """Retorna un iterador para recorrer la cola sin usar listas"""
        self.iterador = self.frente
        return self

    def __next__(self):
        """Retorna el siguiente elemento en la iteración."""
        if self.iterador is None:
            raise StopIteration
        dato = self.iterador.dato
        self.iterador = self.iterador.siguiente
        return dato

```

Figura 3: Implementación de la clase `Cola`.

Paquete services: Lógica de Negocio

Este paquete desacopla la lógica de negocio de la interfaz de usuario.

- **turno_service.py**: Es el "cerebro" de la aplicación. Contiene la clase `TurnoService`, que maneja la instancia de la Cola, registra nuevos pacientes, los atiende, calcula los tiempos de espera y genera los objetos de `ReporteEstadisticas`.
- **graphviz_service.py**: Encapsula toda la lógica relacionada con la generación de gráficos. Recibe datos de la capa de servicio y los traduce a diagramas de Graphviz, generando los archivos de imagen correspondientes para la cola, las fichas y las estadísticas.

```
# src/services/turno_service.py
"""
Módulo que maneja la lógica de negocio del sistema de turnos médicos.
"""
from models.cola import Cola
from models.paciente import Paciente

# Se importan las constantes y las clases de reporte
from utils.constants import (
    ESPECIALIDAD_1, TIEMPO_ATENCION_1,
    ESPECIALIDAD_2, TIEMPO_ATENCION_2,
    ESPECIALIDAD_3, TIEMPO_ATENCION_3,
    ESPECIALIDAD_4, TIEMPO_ATENCION_4
)
from models.reporte import ReporteEstadisticas, NodoEstadistica

class TurnoService:
    """
    Servicio que gestiona la lógica de turnos médicos.
    """

    def __init__(self):
        """Inicializa el servicio de turnos."""
        self.cola_turnos = Cola()

    def registrar_turno(self, nombre, edad, especialidad):
        """Registra un nuevo turno en la cola."""
        if not nombre or not nombre.strip():
            raise ValueError("El nombre no puede estar vacío")

        if not isinstance(edad, int) or edad <= 0 or edad > 120:
            raise ValueError("La edad debe ser un número válido entre 1 y 120")

        # Validación sin usar diccionarios o listas
        if especialidad not in (ESPECIALIDAD_1, ESPECIALIDAD_2, ESPECIALIDAD_3, ESPECIALIDAD_4):
            raise ValueError(f"Especialidad '{especialidad}' no válida")

        paciente = Paciente(nombre.strip(), edad, especialidad)
        self.cola_turnos.encolar(paciente)
        return paciente

    def atender_siguiente_paciente(self):
        """Atiende al siguiente paciente en la cola."""
        if self.cola_turnos.esta_vacia():
            raise IndexError("No hay pacientes en espera")
        return self.cola_turnos.desencolar()

    def ver_siguiente_paciente(self):
        """Muestra el siguiente paciente sin atenderlo."""
        if self.cola_turnos.esta_vacia():
            return None
        return self.cola_turnos.ver_frente()

    def obtener_todos_turnos(self):
        """
        Devuelve la cola iterable directamente, no una lista.
        """
        return self.cola_turnos

    def calcular_tiempo_espera_paciente(self, posicion):
        """
        Calcula el tiempo de espera iterando sobre la cola, sin convertirla a lista.
        """
        if posicion < 0 or posicion >= self.cola_turnos.obtener_tamaño():
            return 0

        tiempo_espera = 0
        contador_actual = 0

        # Itera sobre la cola y se detiene cuando llega a la posición deseada
        for paciente in self.cola_turnos:
            if contador_actual < posicion:
                tiempo_espera += self.obtener_tiempo_atencion(paciente.especialidad)
                contador_actual += 1
            else:
                break # Deja de sumar cuando alcanza la posición

        return tiempo_espera
```

Figura 4: Fragmento del TurnoService.

Paquete gui: Interfaz de Usuario

Contiene la implementación de la ventana principal de la aplicación.

- **interfaz.py:** Define la clase `InterfazTurnos`, que construye todos los widgets de Tkinter (botones, etiquetas, campos de texto, etc.). Maneja los eventos del usuario (clics de botón), llama a los métodos de los servicios correspondientes y actualiza la pantalla para reflejar los cambios en el estado del sistema.

```
# src/gui/interfaz.py
"""
Módulo que implementa la interfaz gráfica del sistema de turnos médicos usando Tkinter.
"""

import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
from PIL import Image, ImageTk
import os
import threading

from services.turno_service import TurnoService
from services.graphviz_service import GraphvizService
from models.reporte import ReporteEstadisticas
from utils.constants import (
    VENTANA_TITULO, VENTANA_ANCHO, VENTANA_ALTO,
    ESPECIALIDAD_1, ESPECIALIDAD_2, ESPECIALIDAD_3, ESPECIALIDAD_4,
    COLOR_FONDO, COLOR_PRIMARIO, COLOR_SECUNDARIO,
    COLOR_EXITO, COLOR_ERROR, FUENTE_TITULO, FUENTE_NORMAL,
    MENSAJE_COLA_VACIA, MENSAJE_PACIENTE_ATENDIDO,
    MENSAJE_ERROR_CAMPOS, MENSAJE_ERROR_EDAD
)

class InterfazTurnos:
    """
    Clase que maneja la interfaz gráfica del sistema de turnos médicos.
    """

    def __init__(self):
        """Inicializa la interfaz gráfica."""
        self.turno_service = TurnoService()
        self.graphviz_service = GraphvizService()

        self.root = tk.Tk()
        self.root.title(VENTANA_TITULO)
        self.root.geometry(f'{VENTANA_ANCHO}x{VENTANA_ALTO}')
        self.root.configure(bg=COLOR_FONDO)
        self.root.resizable(True, True)

        self._configurar_estilos()

        self.nombre_var = tk.StringVar()
        self.edad_var = tk.StringVar()
        self.especialidad_var = tk.StringVar(value=ESPECIALIDAD_1)

        self._crear_widgets()
        self._actualizar_display()

    def _configurar_estilos(self):
        """Configura los estilos para los widgets de ttk."""
        self.style = ttk.Style(self.root)
        self.style.theme_use('clam') # Usar un tema que permita personalización

        # Estilo general para Frames y Labels
        self.style.configure('TFrame', background=COLOR_FONDO)
        self.style.configure('TLabel', background=COLOR_FONDO, foreground='black', font=FUENTE_NORMAL)
        self.style.configure('Titulo.TLabel', background=COLOR_FONDO, foreground=COLOR_PRIMARIO, font=FUENTE_TITULO)

        # Estilo para LabelFrame (recursos con título)
        self.style.configure('TLabelFrame', background=COLOR_FONDO, bordercolor=COLOR_PRIMARIO, font=FUENTE_NORMAL)
        self.style.configure('TLabelFrame.TLabel', background=COLOR_FONDO, foreground=COLOR_PRIMARIO, font=FUENTE_NORMAL)

        # Estilo para Botones
        self.style.configure('TButton', font=FUENTE_NORMAL, padding=5, borderwidth=1)
        self.style.map('TButton',
            foreground=[('pressed', 'white'), ('active', 'white')],
            background=[('pressed', 'disabled', COLOR_SECUNDARIO), ('active', COLOR_SECUNDARIO)])

        self.style.configure('Primary.Button', background=COLOR_PRIMARIO, foreground='white')
        self.style.configure('Success.Button', background=COLOR_EXITO, foreground='white')
        self.style.configure('Danger.Button', background=COLOR_ERROR, foreground='white')

        # Estilo para Combobox
        self.style.map('TCombobox', fieldbackground=[('readonly', 'white')])
        self.style.map('TCombobox', selectbackground=[('readonly', COLOR_PRIMARIO)])
        self.style.map('TCombobox', selectforeground=[('readonly', 'white')])

        # Estilo para Notebook (pestanas)
        self.style.configure('TNotebook', background=COLOR_FONDO, borderwidth=1)
        self.style.configure('TNotebook.Tab', background=COLOR_FONDO, foreground=COLOR_SECUNDARIO,
            font=FUENTE_NORMAL, padding=[10, 5], borderwidth=1)
        self.style.map('TNotebook.Tab',
            background=[('selected', COLOR_PRIMARIO)],
            foreground=[('selected', 'white')])

    def _crear_widgets(self):
        """Crea todos los widgets de la interfaz."""
        # Frame principal
        main_frame = ttk.Frame(self.root, style='TFrame')
        main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        # Título
        titulo_label = ttk.Label(main_frame, text=VENTANA_TITULO, style='Titulo.TLabel')
        titulo_label.pack(pady=(0, 20))

        # Frame superior para formulario
        form_frame = ttk.LabelFrame(main_frame, text="Registrar Nuevo Turno", padding=20)
        form_frame.pack(fill=tk.X, pady=(0, 10))

        # Campos del formulario
        self._crear_formulario(form_frame)

        # Frame para botones principales
        button_frame = ttk.Frame(main_frame)
        button_frame.pack(fill=tk.X, pady=(0, 20))

        self._crear_botones_principales(button_frame)

        # Frame para visualización
        display_frame = ttk.Frame(main_frame)
        display_frame.pack(fill=tk.BOTH, expand=True)

        self._crear_area_visualizacion(display_frame)
```

Figura 5: Fragmento de la clase `InterfazTurnos`.

Conclusiones

Conclusiones Técnicas

- ▶ La implementación de una **Cola dinámica desde cero** demostró ser un método efectivo para cumplir con la restricción de no usar estructuras de datos nativas, forzando una comprensión profunda del manejo de memoria y punteros.
- ▶ La **arquitectura en capas** (modelos, servicios, GUI) fue crucial para mantener el código organizado, desacoplado y fácil de depurar, especialmente al resolver la importación circular.
- ▶ La integración de **Tkinter y Graphviz** en un mismo sistema es viable y potente, aunque requiere una gestión cuidadosa de los hilos (**threading**) y las actualizaciones de la GUI (**root.after**) para mantener una experiencia de usuario fluida y sin bloqueos.

Apéndices

Código Fuente Completo

El código fuente completo y funcional del proyecto está disponible en el siguiente repositorio de GitHub, incluyendo todos los módulos descritos en este manual.

https://github.com/WilsonJrSantos/IPC2_Practica2_201907179